USING THE REPOSITORY ABSTRACTION FOR DATA ACCESS

The `BookService` class relies on a `BookRepository` object to retrieve and save books. The domain layer should be unaware of how data is persisted, so `BookRepository` should be an interface to decouple the abstraction from the actual implementation. Create a `BookRepository` interface in the `com.polarbookshop.catalogservice.domain` package to define the abstraction for accessing book data.

Listing 3.6 The abstraction used by the domain layer to access data

```
package com.polarbookshop.catalogservice.domain;

import java.util.Optional;

public interface BookRepository {
  Iterable<Book> findAll();
  Optional<Book> findByIsbn(String isbn);
  boolean existsByIsbn(String isbn);
  Book save(Book book);
  void deleteByIsbn(String isbn);
}
```

While the repository interface belongs to the domain, its implementation is part of the persistence layer. We'll add a data persistence layer using a relational database in chapter 5. For now, it's enough to add a simple in-memory map to retrieve and save books. You can define the implementation in an `InMemoryBookRepository` class, located in a new `com.polarbookshop.catalogservice.persistence` package.

Listing 3.7 In-memory implementation of the BookRepository interface